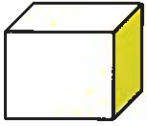


Contents



Programs

- **COIN315-Data Structures**
- **COIN325-Java Programming**
- **COIN332-Applied Networking**
- **COIN333-Applied Systems**



Ethic Papers

- **COIN315-Data Structures**
- **COIN325-Java Programming**
- **COIN480-Artificial Intelligence**



Presentations

- **COIN495-System Analysis &
Design**
- **ARV161- Visual Communication
Media**



Programs:

- **COIN315-Data Structures**
- **COIN325-Java Programming**
- **COIN332-Applied Networking**
- **COIN333-Applied Systems**

CLASS: COIN315-Data Structures

GRADE: A

TOPIC: Ordered Multi-List



program

COIN315-Data Structures

Ordered Multi-List

Data Structures

The Ordered Multi-List is a container type class that will hold integers. The internal data structure of this is a singly-linked, linear list.

```

/*****
*FILENAME:      OrderedMultiList.cpp
*PROGRAMMER:    Rebecca Randall
*PURPOSE:       implementation for an ordered Multi-List holding ints
*****/

#include "OrderedMultiList.h"
#include <iostream>
using namespace std;

/*****
*PURPOSE:       constructor that initializes variables.
*****/
OrderedMultiList::OrderedMultiList(){
    head = NULL;//set the head to point to the end
    sizeTot = 0;//total number of all integers, including repeating ones,
    uniqueSizeTot = 0;//total number of integers in the array, excluding repeating ones
    instances = 0;//number of instances a number is in a list
} //END of constructor OrderedMultiList()

/*****
*PURPOSE:       to check to see if the list is empty.
*****/
bool OrderedMultiList::empty() const{
    return (head==NULL);
} //END of empty()

/*****
*PURPOSE:       to clear the list.
*****/
void OrderedMultiList::clear(){
    Node *current;
    while(head != NULL){
        current = head;//assigns current to the first node
        head = head -> next;//moves head to the next node
        delete current;
    }
} //END of clear()

/*****
*PURPOSE:       to copy the list.
*****/
void OrderedMultiList::copy(const OrderedMultiList& listToBeCopied){
    Node *newNode;//pointer to new nodes to be created
    Node *current;//goes thru the listToBeCopied
    Node *last;//sets the links up between the nodes of the new list
    if (head != NULL){if list is not empty, clear it.
        clear();
    }
}

```

```

    }
    if(listToBeCopied.head == NULL){//listToBeCopied is empty
        head = NULL;
        last = NULL;
        uniqueSizeTot = 0;
        sizeTot = 0;
    }
    else{//not empty{
        current = listToBeCopied.head; //current points to the list to be copied.
        uniqueSizeTot= listToBeCopied.uniqueSizeTot;//sets the same amount of
                                                    unique numbers
        sizeTot = listToBeCopied.sizeTot;//sets the same amount of total numbers in
                                        the list

        //copy the first node
        head = new Node;
        head->value = current ->value;
        head->counter = current ->counter;
        head->next = NULL;

        last = head;
        current = current->next;
        //copy until NULL
        while(current != NULL){
            newNode = new Node;
            newNode -> value = current->value;
            newNode -> counter= current ->counter;
            newNode ->next = NULL;

            //sets the links up between the nodes
            last->next = newNode;
            last = newNode;

            //points at the next node in the original list
            current = current ->next;
        }//END of while
    }//END of else
} //END of COPY()

/*****
*PURPOSE:          to destroy list of vals.          *
*****/
OrderedMultiList::~~OrderedMultiList(){
    if(!empty()){
        clear();
        uniqueSizeTot = 0;
        sizeTot= 0;
        instances = 0;
    }
} //END of DESTRUCTOR

```

```

/*****
*PURPOSE:          to overload the assignment operator.
*****/
const OrderedMultiList& OrderedMultiList::operator= (const OrderedMultiList &listToBeCopied){
    if(this != &listToBeCopied){//check so make sure it's not copying itself
        copy(listToBeCopied);
    }
    return *this;
}
//END of DESTRUCTOR

/*****
*PURPOSE:          returns how many different numbers are in the list (no repetitions)
*****/
int OrderedMultiList::uniqueSize()const{
    return uniqueSizeTot;
}
//END of uniqueSize()

/*****
*PURPOSE:          returns the total number of items in the list including repetition
*****/
int OrderedMultiList::size()const {
    return sizeTot;
}
//END of size()

/*****
*PURPOSE:          finds a specific value with in the list and return true iff the list
*                  contains the val.
*****/
bool OrderedMultiList::find(int val){
    if(uniqueSizeTot > 0){
        Node *current = head;//define the pointer and set it to head
        bool exist = false;
        while(current != NULL && !exist){
            if(current->value == val){//val found{
                instances = current->counter;
                exist = true;
            }
            else
                current = current->next;//change point to the next node
        }
        //END while
        return exist;
    } else return false;
}
//END of find()

/*****
*PURPOSE:          returns number of instances val is in the list.
*****/
int OrderedMultiList::count(int val){
    if(head==NULL)//list is empty
        return 0;
    else if(find(val))//if found is true
        return instances;
    else
        return 0;//if not in the list return 0
}
//END of the count()

```

```

/*****
*PURPOSE:      prints out the content of the list in order.
*****/
void OrderedMultiList::print()const{
    if (uniqueSizeTot > 0){
        Node *current = head;
        while (current != NULL){
            //if head is not NULL then print otherwise do nothing
            cout << current->counter << " " << current->value << endl;
            current = current->next;
        }
    }
} //END of the print()

//mutators

/*****
*PURPOSE:      inserts item val count number of times.
*              returns false if count less than 1 or if val would not fit.
*****/
bool OrderedMultiList::insert(int val, int ct){
    if(ct >= 1){
        Node *current; //pointer to the current node
        Node *previous; //pointer to the previous node
        Node *newNode; //pointer to the newly created node

        newNode = new Node; //creates a new node
        if (newNode == NULL) //checks if out of memory
            return false;

        //assigns the values to node
        newNode->value = val;
        newNode->counter = ct;
        newNode->next = NULL;

        if (empty()) //checks if empty{
            head = newNode;
            newNode->next = NULL;

            //update counters
            uniqueSizeTot++;
            sizeTot += ct;
            return true;
        }
        else{ //not empty
            current = head; //assigns current to the first node
            previous = head;
            bool exist = false;

            while(current != NULL && !exist){ //searches the list
                if(current->value == val){ //val exists
                    current->counter += ct;
                    sizeTot += ct;
                    return true;
                }
            }
        }
    }
}

```

```

        }else if(current->value > val){//passed the position to insert
            val
            exist = true;
        else { //adjusts pointers to the next node
            previous = current;
            current = current ->next;
        }
    }//END of while loop

    if(current == NULL){ //insert at the end
        previous ->next = newNode;
        newNode -> next = NULL;
        uniqueSizeTot++;
        sizeTot+=ct;
        return true;
    } else if(current==head){ //insert at the beginning
        newNode->next=head;
        head = newNode;
        uniqueSizeTot ++;
        sizeTot+=ct;
        return true;
    }
    if(current->value > val){ //insert in the middle
        previous ->next = newNode;
        newNode -> next = current;
        uniqueSizeTot++;
        sizeTot+=ct;
        return true;
    }

    return false;
} //End of else, not empty
return false;
} //END if ct is greater than 1
return false;
} //END of insert() with a ct
/*****
*PURPOSE:    inserts item val and returns false if count less than 1 or
*            if val would not fit.
*****/
bool OrderedMultiList::insert(int val){
    return(insert(val,1));
} //END of insert()

/*****
*PURPOSE:    removes 1 instance of val from list and returns true iff val was in list
*****/
bool OrderedMultiList::remove(int val){
    Node *current;
    Node *previous;
    bool exist;
    if(head == NULL){//the list is empty
        return false;
    }
    else{
        current = head;
        exist = false;

```

```

while(current != NULL && !exist){//check if val exist
    if(current->value == val)
        exist = true;
    else{
        previous = current;
        current = current->next;
    }
}

if(current == NULL)//val is not in the list
    return false;
else{
    if(head==current){//val is first in the list
        if(current->counter > 1){
            current->counter--;
            sizeTot--;
            return true;
        }else{
            head = head -> next;
            sizeTot-=current->counter;
            uniqueSizeTot--;
            delete current;
            return true;
        }
    }else{ //val is between other nodes
        if(current->counter > 1){
            current->counter--;
            sizeTot--;
            return true;
        }else{
            previous->next = current->next;
            sizeTot-=current->counter;
            uniqueSizeTot--;
            delete current;
            return true;
        }
    }
}
} //END else between nodes
} //END else val is in the list
} //END else not empty
} //END OF remove()

/*****
*PURPOSE:      :removes all instances of val from list and returns the number of
*              items removed which could be 0.
*****/
int OrderedMultiList::removeAll(int val){
    Node *current;
    Node *previous;
    bool exist;
    int valCounter = 0;//number of instances a val was removed

    if(head == NULL)//the list is empty
        return false;
    else{

```

```

current = head;
exist = false;

while(current != NULL && !exist){ //check if val exist
    if(current->value == val)
        exist = true;
    else {
        previous = current;
        current = current->next;
    }
}

if(current == NULL)//val is not in the list
    return 0;
else{
    if(head==current){ //val is first in the list
        head = head -> next;
        valCounter = current->counter;
        sizeTot=current->counter;
        uniqueSizeTot--;
        delete current;
    }
    else{ //val is between other nodes
        previous->next = current->next;
        valCounter = current->counter;
        sizeTot=current->counter;
        uniqueSizeTot--;
        delete current;
    }
}
return valCounter;
} //END OF remove()

```

CLASS: COIN325-Java Programming

GRADE: A

TOPIC: Employee Payroll System



program

COIN325-JAVA PROGRAMMING

Employee Payroll System

Java Programming

This program would add a new employee to a payroll system based on the type of employee they were. In additions it would calculate their salary.

```

/*****
*FILENAME:      PayableInterfaceTest.java
*PROGRAMMER:    Rebecca Randall
*PURPOSE:       test the java program by inserting another employee
*****/
import java.util.Scanner;

public class PayableInterfaceTest{
    public static void main(String args[]){
        Scanner input = new Scanner(System.in);

        Date birthHourly = new Date(6,15,1944);
        Date birthCommission = new Date(12,29,1960);
        Date birthBase = new Date(3,2,1965);

        Employee EmployeeObjects[] = new Employee[3];
        EmployeeObjects[0]= new HourlyEmployee("Bri","Anna",birthHourly,"321-
        21-4321",16.75,40);
        EmployeeObjects[1]= new CommissionEmployee("Rob","Bert",birthCommission,
        "987-87-9876",10000,0.06);
        EmployeeObjects[2]= new BasePlusCommissionEmployee
        ("Wayne","Nicole",birthBase, "456-56-5678", 5000,0.04,300.00);

        System.out.println("Employees processed polymorphically:\n");
        for (Employee currentPayable: EmployeeObjects){
            System.out.printf("%s\n", currentPayable.toString());

            System.out.printf("%s: $%,.2f\n", "payment due",
            currentPayable.getPaymentAmount());
        }

        System.out.print("Enter the current month (1-12): ");
        int curMonth = input.nextInt();

        System.out.print("Employees processed polymorphically:");
        for (Employee currentPayable: EmployeeObjects){
            System.out.printf("\n\n%s\n", currentPayable.toString());

            if(currentPayable instanceof BasePlusCommissionEmployee){
                BasePlusCommissionEmployee object =
                (BasePlusCommissionEmployee) currentPayable;
                double oldBaseSalary = object.getBaseSalary();
                object.setBaseSalary(1.10*oldBaseSalary);
                System.out.printf("new base salary with 10%% increase is:
                $%,.2f\n",object.getBaseSalary());
            }

            System.out.printf("%s: $%,.2f", "earned",
            currentPayable.getPaymentAmount());
        }
    }
}
```

program

```

        if((currentPayable instanceof CommissionEmployee) && !(currentPayable instanceof
        BasePlusCommissionEmployee) && (birthCommission.getMonth()==curMonth))
            System.out.print(" plus $100.00 birthday bonus\n");
        else if((currentPayable instanceof HourlyEmployee) &&
        (birthHourly.getMonth()==curMonth))
            System.out.print(" plus $100.00 birthday bonus\n\n");
        else if((currentPayable instanceof BasePlusCommissionEmployee) &&
        (birthBase.getMonth()==curMonth))
            System.out.print(" plus $100.00 birthday bonus\n\n");
    }
    System.out.print("\n\n");
    for(int i=0; i<EmployeeObjects.length;i++)
        System.out.printf("Employee %d is a
        %s\n",i,EmployeeObjects[i].getClass().getName());
    }
}

```

```

/*****
*FILENAME:      Payable.java
*PROGRAMMER:    Rebecca Randall
*PURPOSE:       calculates the amount to pay an employee
*****/

```

```

public interface Payable
{
    double getPaymentAmount();
}

```

```

/*****
*FILENAME:      Employee.java
*PROGRAMMER:    Rebecca Randall
*PURPOSE:       sets up an employee
*****/

```

```

public abstract class Employee implements Payable
{
    private String firstName;
    private String lastName;
    private String socialSecurityNumber;
    private Date birthDate;

    public Employee(String first, String last, Date dateOfBirth, String ssn) {
        firstName = first;
        lastName = last;
        birthDate = dateOfBirth;
        socialSecurityNumber = ssn;
    }

    public void setFirstName(String first){
        firstName = first;
    }

    public String getFirstName(){
        return firstName;
    }
}

```

```

    public void setLastName(String last){
        lastName = last;
    }

    public String getLastName(){
        return lastName;
    }

    public void setSocialSecurityNumber(String ssn){
        socialSecurityNumber = ssn;
    }

    public String getSocialSecurityNumber(){
        return socialSecurityNumber;
    }

    public String toString(){
        return String.format("%s %s\nsocial security number: %s\nbirth date: %s",getFirstName(),getLastName(), getSocialSecurityNumber(),birthDate);
    }
}

```

```

/*****
*FILENAME:      HourlyEmployee.java
*PROGRAMMER:    Rebecca Randall
*PURPOSE:       sets up an employee that is paid hourly
*****/

```

```

public class HourlyEmployee extends Employee
{
    private double wage;
    private double hours;

    public HourlyEmployee(String first, String last, Date birth, String ssn, double    hourlyWage, double
hoursWorked) {
        super(first, last, birth,ssn);
        setWage(hourlyWage);
        setHours(hoursWorked);
    }

    public void setWage(double hourlyWage){
        wage = (hourlyWage < 0.0) ? 0.0 : hourlyWage;
    }

    public double getWage(){
        return wage;
    }

    public void setHours(double hoursWorked){
        hours=((hoursWorked>=0.0)&&(hoursWorked<=168.00))?hoursWorked:0.0;
    }

    public double getHours(){
        return hours;
    }
}

```

```

//interface payable
public double getPaymentAmount(){
    if(getHours()<=40)
        return getWage()*getHours();
    else return 40*getWage() + (getHours()-40)* getWage()*1.5;
}

public String toString(){
    return String.format("hourly employee: %s\n%s: $%,.2f; %s: %%,.2f",
super.toString(), "hourly wage", getWage(), "hours worked", getHours());
}
}

```

```

/*****
*FILENAME:      CommissionEmployee.java
*PROGRAMMER:    Rebecca Randall
*PURPOSE:       sets up an employee that is paid by commission
*****/

```

```

public class CommissionEmployee extends Employee
{
    private double grossSales;
    private double commissionRate;

    public CommissionEmployee(String first, String last, Date birth, String ssn, double
        sales, double rate){
        super(first, last, birth, ssn);
        setGrossSales(sales);
        setCommissionRate(rate);
    }

    public void setCommissionRate(double rate){
        commissionRate=(rate>0.0&&rate<1.0)?rate:0.0;
    }

    public double getCommissionRate(){
        return commissionRate;
    }

    public void setGrossSales(double sales){
        grossSales=(sales<0.0)?0.0:sales;
    }

    public double getGrossSales(){
        return grossSales;
    }

    public double getPaymentAmount(){
        return getCommissionRate()*getGrossSales();
    }

    public String toString(){
        return String.format("%s: %s\n%s: $%,.2f; %s: %%,.2f",
"commission employee", super.toString(),
"gross sales", getGrossSales(),

```

```

        "commission rate", getCommissionRate());
    }
}

```

```

/*****
*FILENAME:      BasePlusCommissionEmployee.java
*PROGRAMMER:    Rebecca Randall
*PURPOSE:       sets up an employee that is paid by commission
*****/

```

```

public class BasePlusCommissionEmployee extends CommissionEmployee
{
    private double baseSalary;

    public BasePlusCommissionEmployee(String first, String last, Date birth, String ssn,
        double sales, double rate, double salary){
        super(first, last, birth, ssn, sales, rate);
        setBaseSalary(salary);
    }

    public void setBaseSalary(double salary){
        baseSalary=(salary<0.0)?0.0:salary;
    }

    public double getBaseSalary(){
        return baseSalary;
    }

    public double getPaymentAmount(){
        return baseSalary +(getCommissionRate()*getGrossSales());
    }

    public String toString (){
        return String.format("%s %s; %s: $%,.2f", "base salaried", super.toString(),
            "base salary", getBaseSalary());
    }
}

```

```

/*****
*FILENAME:      Date.java
*PROGRAMMER:    Rebecca Randall
*PURPOSE:       validate a date
*****/

```

```

public class Date
{
    private int month;
    private int day;
    private int year;

    public Date(int theMonth, int theDay, int theYear){
        month=checkMonth(theMonth);
        year=theYear;
        day=checkDay(theDay);

        System.out.printf("Date object constructor for date %s\n", this);
    }
}

```

```

private int checkMonth(int testMonth){
    if(testMonth>0 && testMonth<=12)
        return testMonth;
    else{
        System.out.printf("Invalid month (%d) set to 1.\n", testMonth);
        return 1;
    }
}

private int checkDay(int testDay){
    int daysPerMonth[]={0,31,28,31,30,31,30,31,31,30,31,30,31};

    if(testDay>0 && testDay <= daysPerMonth[month])
        return testDay;

    if(month==2 && testDay == 29 && (year % 400 == 0 || (year % 4 == 0 && year%100 != 0)))
        return testDay;

    System.out.printf("Invalid day (%d) set to 1.", testDay);
    return 1;
}

public int getMonth(){
    return month;
}

public String toString(){
    return String.format("%d/%d/%d",month,day,year);
}
}

```

CLASS: COIN332-Applied Networking

GRADE: A

TOPIC: Christmas Countdown



program

COIN332-APPLIED NETWORKING

Christmas Countdown

Applied Networking

Based on a future Christmas date given, the clock will begin its countdown until it is reached.

```

/*****
*FILENAME:          randall_countdown.html
*PROGRAMMER:       Rebecca Randall
*PURPOSE:          countdown to a specified Christmas date
*****/

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" >
<head>
  <title>Countdown to Christmas</title>
  <script type="text/javascript">

    //UPDATE COUNTDOWN FUNCTION
    var dayArrived;
    var timer = false;

    function runApp (){
      if(timer)
        clearTimeout(dayArrived);
      timer = false;
      countdown();
    }

    function countdown(){
      //TODAY'S DATE AND SPECIAL DAY WE WANT TO REACH
      var today = new Date();
      var SpecialDay = new Date("December 25 2007 0:00:01");

      //TOTAL MICROSECONDS
      msPerDay = 24 * 60 * 60 * 1000 ;//24 hours 60 minutes 60 seconds 1000 ms
      msPerHour= 60 * 60 * 1000;
      msPerMin = 60 * 1000 ;
      msPerSec = 1000 ;

      /*The time remaining is calculated using miliseconds.*/

      //THIS CALCULATES THE TOTAL DAYS LEFT FOR CHRISTMAS
      var daysLeft = Math.floor((SpecialDay - today) / msPerDay);

      //Total hours left based on the days
      var totHours_d = 24 * daysLeft;

      //Total minutes left based on the days
      var totMins_d = 24 * 60 * daysLeft;

      //THIS CALCULATES THE TOTAL HOURS LEFT FOR CHRISTMAS
      var hoursLeft = Math.floor((SpecialDay - today)/ msPerHour) - totHours_d;
  
```

program

```

//Total minutes left based on the days
var totMin_d = 24 * 60 * hoursLeft;

//Total minutes left based on the hours
var totMin_h = 60 * hoursLeft;

//Total seconds left based on the days
var totSec_d = 24 * 60 * 60 * daysLeft;

//Total seconds left based on the hours
var totSec_h = 60 * 60 * hoursLeft;

//THIS CALCULATES THE TOTAL MINUTES LEFT FOR CHRISTMAS
var minsLeft = Math.floor((SpecialDay - today)/ msPerMin) - (totMins_d +
totMin_h);

//Total seconds left based on the minutes
var totSec_m = 60 * minsLeft;

//THIS CALCULATES THE TOTAL MINUTES LEFT FOR CHRISTMAS
var secsLeft = Math.round((SpecialDay - today) / msPerSec) - (totSec_d +
totSec_h + totSec_m);

//THIS DISPLAYS THE TOTAL DAYS, HOURS, MINUTES, and
SECONDS LEFT FOR SPECIAL DAY
Days.innerText= " Days";
DaysText.innerText = daysLeft;
Hours.innerText = " Hours";
HoursText.innerText = hoursLeft;
Mins.innerText = " Minutes";
MinsText.innerText = minsLeft;
Secs.innerText = " Seconds";
SecsText.innerText = secsLeft;

dayArrived = setTimeout("countdown()",1000);
timer = true;

//checks to see if there is less than one day left
if(daysLeft < 0){
    clearTimeout(dayArrived);
    timer = false;
    document.write('<body background="greeting.jpg"
bgcolor="#FFCC33" style="background-repeat: no-repeat;
background-position: 50% 30px;">')
}
}
</script>
</head>
<body onLoad="runApp()" style="background:url(countdown.jpg);background-repeat: no-repeat;
background-position: 50% 30px;">
    <div align="center"> <br><br><br><br><br><br><br>
    <p><font size="+4" color="gold" id="DaysText"></font> <font size="+2"
color="gold" id="Days"></font></br>
    <font SIZE="+4" COLOR="gold" id="HoursText"></font><font SIZE="+2"
COLOR="gold" id="Hours"></font></br>
    <font SIZE="+4" COLOR="gold" id="MinsText"></font> <font SIZE="+2"

```

program

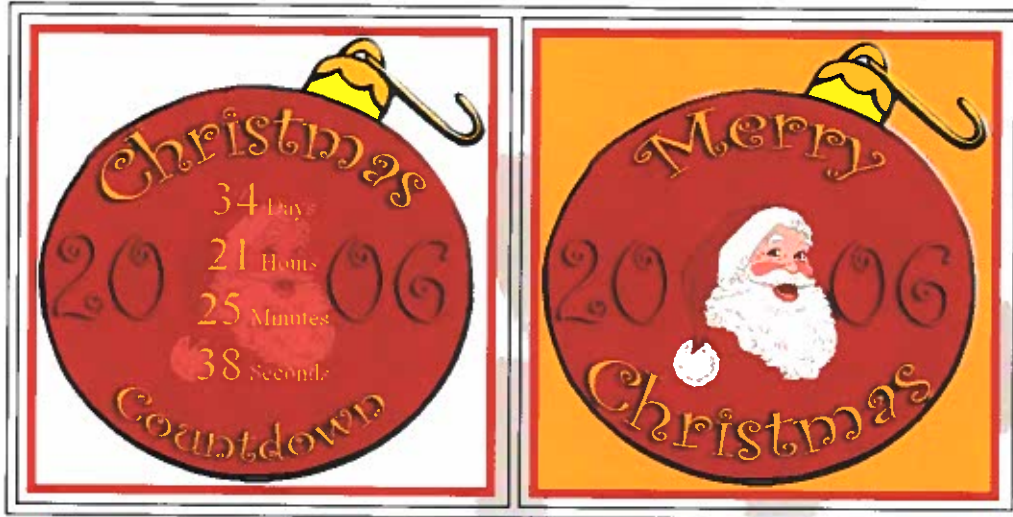
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100

```

COLOR="gold" id="Mins"></font></br>
    <font SIZE="+4" COLOR="gold" id="SecsText"></font> <font SIZE="+2"
COLOR="gold" id="Secs"></font>
    </p>
</div>
</body>
</html>

```

Screen Shots



Program

LINKED LIST IMPLEMENTATION

CLASS: COIN333-Applied Systems

GRADE: A

TOPIC: Y86 Functions



Program

COIN333-Applied Systems

Y86 Functions

Applied Systems

The following are a set of Y86 codes that perform different tasks as required by the instructor Gary Underwood.

```

/*****
*FILENAME:          problem-1..ys
*PROGRAMMER:       Rebecca Randall
*PURPOSE:          A Y86 function called initArr that takes three parameters on a
*                  stack, the address of an array of integers, an integer representing
*                  the size of the array, and an integer for the initial value. It then
*                  initializes all spots in the array to that third parameter.
*****/

```

```

irmovl $0x400, %esp
irmovl $1, %edx      #increment for my counter
irmovl $0, %esi      #counter to reach Mult2
rrmovl Mult1, %eax   #copy Mult1
rrmovl Mult2, %edi   #copy Mult2

call Mult

halt

Mult:      .pos 0x100
           pushl %ebp
           subl %edi, %esi      #checking if either mult2 is zero
           je Zero
           irmovl $0, %esi      #reset esi incase mult2 wasn't zero
           Test: subl %eax, %esi #checking if mult1 is zero
           jne NotZero

Zero:      irmovl $0, %ebx      #set value for ans to zero
           jmp Done            #jump to the final set, set ANS value

NotZero:
           irmovl $0, %esi      #reset esi to zero
           rrmovl %eax, %ebp     #copy Mult1 into a register
           subl %esi, %ebp      #check sign

           jle Else             #less than zero
           jmp Loop

Else:
           irmovl $0, %esi      #reset esi to zero
           subl %eax, %esi      #converted Mult1 from neg to pos
           rrmovl %esi, %ebp     #copy converted value to a register
           irmovl $0, %esi      #reset esi to zero

Loop:      rrmovl %edi, %ecx
           subl %esi, %ecx      #while loop condition checker
           jle After            #condition Mult2 <=0

```

program

CIN 333 - APPLIED SYSTEMS


```

jne NotZero

Zero:  irmovl $0, %ebx #set value for ans to zero
      jmp Done        #jump to the final set, set ANS value

NotZero:
irmovl $0, %esi          #reset esi incase mult1 wasn't zero
subl    %esi, %eax        #check sign of Mult1

jle Else                  #less than zero
jmp Loop

Else:
      irmovl $0, %esi          #reset esi to zero
      subl %eax, %esi          #converted Mult1 from neg to pos
      rrmovl %esi, %eax        #copy converted value
      irmovl $0, %esi          #reset esi to zero

Loop:  rrmovl %edi, %ecx
      subl %esi, %ecx          #while loop condition checker
      jle After                #condition Mult2 <=0
      addl %eax, %ebx          #add mult1 to ans
      addl %edx, %esi          #increment counter
      jmp Loop

After:
      mrmovl 12(%ebp), %eax
      subl    %esi, %eax        #check sign

      jge Done                #greater than zero
      rrmovl %ebx, %eax        #copy the final answer
      subl %eax, %ebx          #convert final answer from + to -
      subl %eax, %ebx
      Done:
            rmmovl %ebx, ANS    #copies final answer into ANS

      popl %ebp

ret

.pos 0x400
.align 4
      #Test values used
ANS:  .long 0 #0 0 0 0 1 -1
Mult1: .long 3 #0 1 -1 0 1 -1
Mult2: .long 2 #0 0 0 1 1 1

```

```

/*****
*FILENAME:      problem-3.ys
*PROGRAMMER:    Rebecca Randall
*PURPOSE:       translate C++ code to Y86
*
*               void main () {
*                   fun(3);
*               }
*               void fun (int x) {
*                   int temp;
*                   foobar (&temp);
*               }
*               void foobar (int *y){
*                   *y=7;
*               }
*****/

```

```

irmovl $0x400, %esp
irmovl $1, %edx      #increment for my counter
irmovl $0, %esi      #counter to reach Mult2
mrmovl Mult1, %eax    #Mult1 to stack (12)
pushl %eax
mrmovl Mult2, %eax    #Mult2 to stack (8)
pushl %eax

irmovl ANS, %ecx      #ANS address on the stack to save final answer

call Mult

halt

Mult:    .pos 0x100
        pushl %ebp

        rrmovl %esp, %ebp
        mrmovl 8(%ebp), %edi  #get Mult2 from the stack
        mrmovl 12(%ebp), %eax  #get Mult1 from the stack
        subl %edi, %esi        #checking if either mult2 is zero
        je Zero
        irmovl $0, %esi        #reset esi incase mult2 wasn't zero
        Test: subl %eax, %esi  #checking if mult1 is zero
        jne NotZero

Zero:    irmovl $0, %ebx #set value for ans to zero
        jmp Done        #jump to the final set, set ANS value

NotZero:
        irmovl $0, %esi        #reset esi incase mult1 wasn't zero
        subl %esi, %eax        #check sign of Mult1

        jle Else              #less than zero
        jmp Loop

Else:
        irmovl $0, %esi        #reset esi to zero
        subl %eax, %esi        #converted Mult1 from neg to pos
        rrmovl %esi, %eax      #copy converted value

```

program

COINTEGRATED SYSTEMS

```

        irmovl $0, %esi          #reset esi to zero

Loop:   rrmovl %edi, %ecx
        subl %esi, %ecx          #while loop condition checker
        jle After                #condition Mult2 <=0
        addl %eax, %ebx          #add mult1 to ans
        addl %edx, %esi          #increment counter
        jmp Loop

After:
        mrmovl 12(%ebp), %eax
        subl %esi, %eax          #check sign

        jge Done                #greater than zero
        rrmovl %ebx, %eax        #copy the final answer

        subl %eax, %ebx          #convert final answer from + to -
        subl %eax, %ebx
        Done:

        mrmovl 16(%ebp), %eax    #get ANS address location
        rrmovl %ebx, %eax        #copies final answer into ANS
        popl %ebp

ret

        .pos 0x400
        .align 4
        #Test values used
ANS:    .long 0 #0 0 0 0 1 -1
Mult1:  .long 3 #0 1 -1 0 1 -1
Mult2:  .long 2 #0 0 0 1 1 1

```

program

3-11-13 01:11:11 3-11-13

Ethic Papers:

- **COIN315-Data Structures**
- **COIN325-Java Programming**
- **COIN480-Artificial Intelligence**

CLASS: COIN315-DATA STRUCTURES

GRADE: B

TOPIC: Ethics & University Admission



ethic paper

COIN315-DATA STRUCTURES

Ethics & University Admission

When applying for a University, every student has the option to select which university they wish to attend by submitting their admission application, but it does not guarantee their acceptance. The decision ultimately comes from the university as to whether or not one has been accepted or denied. Some universities receive far more applications from students wishing to attend their school than they are able to accommodate. So the question becomes how do they determine who will be accepted, denied or placed on a waiting list?

Student's achievements are not the only factor a school uses in their decision; there are several others they take into consideration when reviewing an individual's application. All of these factors help a school decide which student to accept over others since all of them can not be admitted. So if you are lucky enough and have all the qualifications they are looking for, then you are in!! Listed below are some of the factors schools use when determining the admission of a student:

- Gender:** Some schools make their final decision based on one's gender.
- Race:** Another factor that plays a role in determining one's acceptance is their race. Some researchers at the University of Princeton have found that when a school does not take into consideration the race of future students would result in a "sharp decline" in the number of African Americans and Hispanics and no change for the white students (Princeton).
- Resident or non-resident:** Some schools take your residency into account when determining acceptance. Mainly schools of the State tend to lean more towards accepting their State residents over those that are from out-of-state, even if they have higher qualities. Typically there are more spots for in-State students than for out-of-State (CollegeData).
- Income:** Some schools make their decision on one's economical background.
- GPA:** (Grade Point Average) This is a no brainer when it comes to admitting someone to a school, the higher your grades, the better your chances of being selected. Typically you

must have a 3.0 grade point average on a 4.0 scale to be selected; some schools set a higher GPA (Kantrowitz).

- Class Rank:** A student who ranks highest in their class also increases his/her chance of admission.

- Numerous advance placements and Honor Classes:** the fact that one has completed several honor classes and achieved numerous advance placements can also be a factor that greatly aides in the decision of their acceptance. These classes show your hard work and "willingness to challenge yourself" (CollegeData).

- Standardized test:** as with your GPA, your TOEFL, SAT, ACT, GRE, and GMAT test scores play a role in a schools decision to determine if you will become one of their students. Depending on what school you wish to attend, they have different score levels. Some schools set very high standards for admission due to the high volume of student applications, such as Harvard and Yale. "These types of test demonstrate your strengths and weaknesses, compared with those of everyone else taking the same tests" (CollegeData)

- Essays:** there are some schools that require you to write an essay based on a question the school provides. This allows the school to "find out more about you as an individual, what means the most to you, what experiences have affected you the most or what you have to say about your world." A good essay can tip a decision in your favor (CollegeData).

- Personal recommendations:** Letters of recommendation are another plus that can make a difference when ones application is compared with another that it similar.

- Interviews:** The interview is just one of many factors in the admission decision. If one is on the borderline and the student turns out to be impressive during the interview, the interviewer has the authority to write a letter in support.

- Special Talent:** Possessing a special talent that a school wants can easily increase your acceptance.

- Extracurricular info** (sports, performing arts, community action, and leadership activities):

In addition to good grades, the extracurricular activities ones participate in another quality schools look for. "They want individuals who will round out their student body with new interests and talents, while fitting in with the other students" (CollegeData).

So with the factors listed above, Universities every where must eventually determine who will become a student and who will be rejected. But sometimes not always has Universities used fair guidelines when selecting their students. There has been an occasion where the rejection of a student has ended up in a lawsuit. One such case is Grutter vs. Bollinger.

Barbara Grutter was a resident of Michigan who applied to the University of Michigan Law School in 1996 and was rejected. She had a 3.8 GPA and 161 LSAT score. She filed a lawsuit in 1997 stating that the Law School denied her admission mainly due to her race, which violated the Fourth Amendment. She also stated that the university was giving "underrepresented minority groups a significantly greater chance of admissions than White and Asian American applicants with similar credentials" (Wikipedia). The University of Michigan defended their position stating that they used it to provide diversity within their school. But after taking the university to court, the judge O'Connor found that the "law school's interest in obtaining a critical mass of minority students was indeed a tailored use. O'Connor ordered the university to eliminate race as a factor in creating diversity among the students and eventually come up with a "colorblind" policy (Wikipedia).

Another case that ended up in the court system due to rejection of a student based on discrimination was the case in 1997 of Smith vs. The University of Washington School of Law. Once again the issue was using race as a factor in admissions. In this case three white students, Katrina Smith, Angela Rock and Michael Pyle, claimed that the law school "acted unconstitutionally" when they used race as a factor in determining their acceptance. To defend their position, the school had an admissions policy in place that was "designed according to the landmark 1979 Supreme Court Bakke ruling" (So). That ruling stated that "race could be used as one of many factors to achieve diversity in the school" (So). At first, the court ruled in the University's favor but like all court battles, it didn't end there. The plaintiffs were not satisfied with the verdict so they took it to a higher court

which ultimately found the University Of Washington School Of Law at fault.

From a Christian point of view, what is ethically correct in determining the admission of a student is keeping in mind what God would want us to do. God wants everyone to be like him, poses his good nature of being "truthful, loving, holy" (Geisler). So in determining the acceptance of a student to a school, God would want you to determine this based on qualities an individual possesses and not based on the color of one's skin or their gender. It all boils down to do what God would want us to do, and that is treating everyone equally without discriminating.

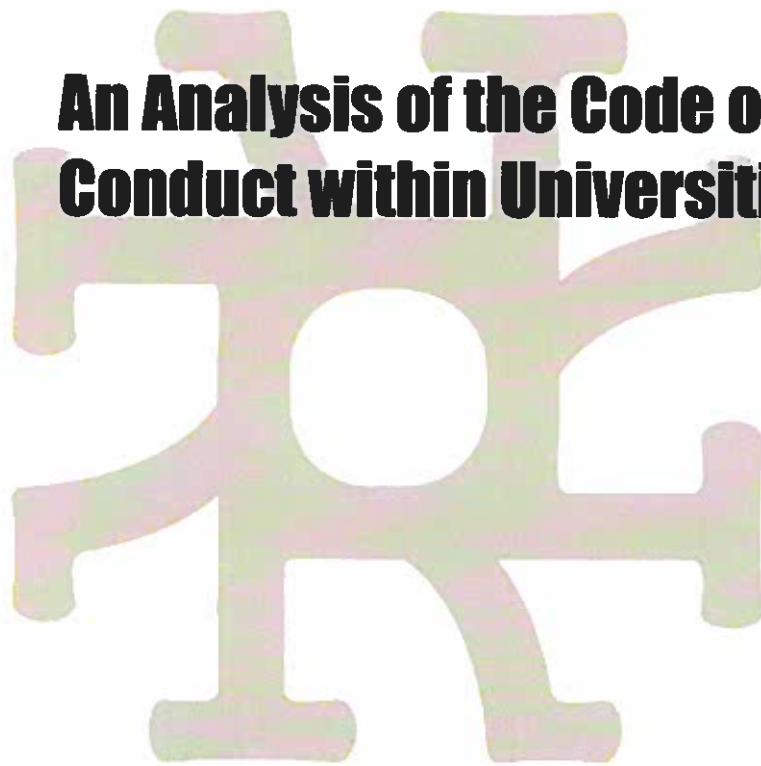
Work Cited

1. CollegeData, Understanding College Selectivity, collegedata.com/cs/content/content_choosearticle_tmpl.jhtml?articleId=10004; Jan.15,07
2. Kantrowitz, Mark; eduPASS "Admission Requirements", <http://www.edupass.org/admissions/requirements.phtml>; Jan 15,07
3. Princeton; News@Princeton, "Study: Ending affirmative action would devastate most minority college enrollment" <http://www.princeton.edu/main/news/archive/S11/80/77123/index.xml>; Jan 16,07
4. Wikipedia, Grutter v. Bollinger http://en.wikipedia.org/wiki/Grutter_v._Bollinger; Jan.20,07
5. So, Jimmy; U-Wire "Case against U. Washington law school declined in Supreme Court" 5/3/01 <http://www.uwire.com/content/topnews053001001.html>; Jan 20,07
6. Geisler, Norman; Christian Ethics "Teaching Format Outline of "Christian Ethics". <http://www.christian-faith.com/ethics/Ethics1.htm> Jan.23,07

CLASS: COIN325-Java Programming

GRADE: A

TOPIC: An Analysis of the Code of
Conduct within Universities



ethic paper

COIN325-JAVA PROGRAMMING

An Analysis of the Code of Conduct

within Universities as it relates to the Internet

The Internet provides rich sources of information as it consists of a global network connecting millions of computers. There are probably over more than 100 countries that are linked into exchanges of data, news and opinions. With this being said, these rich sources of information available on the Internet greatly enhancing the quality of education available to all students. Any student desiring to find, read, or discover new information may do some with the push of a button. But one knows that there is much more information that roams the Internet that a student should not be viewing while accessing the Internet from school grounds. So what guidelines or codes of conduct should a university put in place when it comes to students accessing it from their campus?

Currently, Charleston Southern University gives all its students access to the Internet "free of charge" (CSU) as long as they agree to "act in a considerate and responsible manner" (CSU). They believe this will "enable students to explore thousands of libraries, databases and files throughout the world [and] facilitate diversity and personal growth in technology, informational research skills, and communication skills" (CSU). But one knows there is a "dark side" to the Internet that a University would not want you to access or perform while using their Internet access. So the question becomes: what rules and guidelines must one have to follow while using the Internet provided by their university? In other words, what would be a fare Code of Conduct that would protect the university? There are several guidelines one must follow at Charleston Southern University (CSU). Some of them being obvious that you must not send any emails "that are offensive or threatening, not to interfere with the computer work of others or purposely try to disturb the integrity of the network"(CSU).

I think that the biggest issue is the information or materials that are downloaded or accessed over the Internet and what is done with them. Now a days you can find just about anything over the Internet: songs, movies, software, research papers of any kind of subject matter, program codes, etc. But one knows that you must not download or use them if they are copyrighted and if you have not paid for them. By doing so, one runs the risk of jeopardizing the computer systems if the user

does not take adequate precautions against malicious downloads. In addition, if the downloads are copyrighted, the individual is violating the copyright law which is illegal. So that the University can protect itself against legal actions, it requires each individual to agree upon its guidelines under the Code of Conduct before allowing anyone to use one of its computers to access the Internet (CSU). Now each individual "accept the responsibility for all the material received or sent through/over the network" (CSU). Ultimately the University must protect itself from the unlawful actions that some students may perform over the Internet and from the malicious or destructive attacks to its networks. After all, every University provides their students with the Internet for the purpose of communication, research, and education.

I agree with the guidelines Charleston Southern University makes each student agree upon in order to have access to the Internet on their campus. Now days there are many destructive and illegal things one can do at the push of a button. So to protect their network and protect all other CSU students, the guidelines implemented are adequate. I have to say that most of them are common sense and things you should know better not to do. But they all must be listed so we are all on the same page and they we are all responsible for our actions and conducts done on the Internet.

When comparing the guidelines of Charleston Southern University, in regard to the Code of Conduct when using the Internet, to other universities, such as the American University of Beirut and the University of Delaware, they all seem to represent the same basic guidelines. There was one university, University of Leicester, which seemed to cover the guidelines in greater detail. Maybe because they have come across instances where they felt it was needed due to inappropriate behavior. Like all universities, it allows its faculty, staff and students to make use of the internet as they believe it allows them to exercise their "freedom to engage in academic investigation and scholarly debate" (UL). Their Code of Practices is to make sure that the Internet is used in a manner where all the members of the University will benefit from it (UL). In order to make sure no one violates their guidelines, they retain the right to "monitor a selection of messages and materials sent over its network" (UL) so they can check that their facility is not being abused (UL).

In general, all Universities want you to act responsibly and in accordance with relevant laws,

contractual obligations, and with the highest standards of ethics when it comes to using the Internet. After all, one is attending school to gain an education not to perform activities that are not allowed by a university.

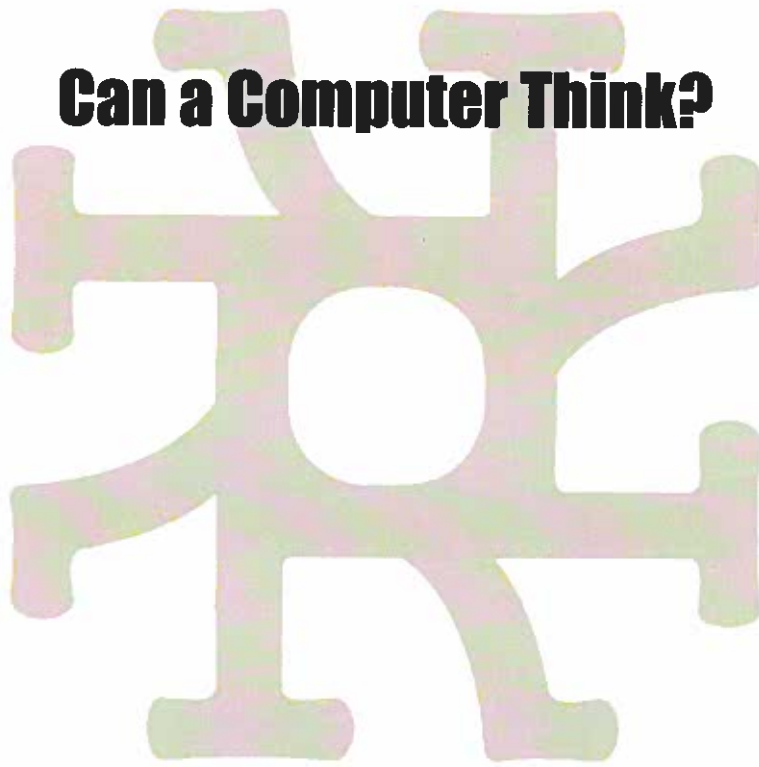
Work Cited

1. CSU-Charleston Southern University "Code of Conduct"
<http://www.csuniv.edu/adminservices/conduct.asp>
2. UD-University of Delaware "Student Guide to University Policies"
<http://www.udel.edu/stuguide/06-07/code.html#respcomp>
3. AUB-American University of Beirut "Code of Conduct for users of Computing Systems and Internet Services" http://www.aub.edu.lb/info/code_aub.html
4. UL-University of Leicester "Internet Code of Practice"
<http://www.le.ac.uk/regulations/computing/internetcode.html>

CLASS: COIN480-Artificial Intelligence

GRADE: A

TOPIC: Can a Computer Think?



Artificial Intelligence

can computers think?

Artificial Intelligence is the study of computer science that focuses on “creating machines that can engage on behaviors that humans consider intelligent”. The area of Artificial Intelligence has been growing everyday, but has it grown far enough for us to say that we have “create[d] a computer that can think?”

Some say “thinking is a function of man's immortal soul. God has given an immortal soul to every man and woman, but not to any other animal or to machines” (Turing). The previous statement seems to indicate that no computer was intended to have a mind of its own. So can an intelligent mind be created to think? An infant is born with a clean slate and as the child grows up, his/her brain is filled with knowledge that he/she has been exposed to. A computer is the same way, it can be trained quickly thru a “set of training data, which have been classified by hand” (Coppin). Some advantages of training a computer is that it “has virtually no operating cost, never forgets what it learns, never calls in sick, retires, or goes on vacation”. Thru adequate training, the machine can think in principle, but not necessarily in the same way humans do. Thinking is very complex. “Even if we someday come to understand all the laws and principles that govern the mind, that doesn't mean that we can duplicate it” (Waldrop). Our brains use a process of thinking and not computation, “thinking is something holistic and emergent-and organic and fuzzy and warm and cuddly and mysterious.” (Waldrop) The distinction between humans and computers is “the essence of humanity isn't reason or logic or any of the other things that computers can do: it's intuition, sensuality, and emotion. So how can a computer think if it does not feel, and how can it feel if it knows nothing of love, anguish, exhilaration, loneliness, and all the rest of what it means to be a living human being” (Waldrop)?

I currently believe that a computer will never be able to think like a human. I view Artificial Intelligence technology as “an extension of our body which allows us to make more rational decisions faster” (Dyess). There are some cases in which a computer system can make better choices than a human. Take for example forecasting the weather which relies on many variables, a

computer system can "more accurately pool all of its knowledge" to make a forecast (Dyess). A system is able to "consider a large amount of information that may not be considered by humans" thus leading to a more accurate prediction (Dyess). There are other areas, such as in medicine, where a computer can aid a doctor in diagnosing a patient but it should never completely replace the doctor's opinion. Humans have intuitions which a system lacks and does not give it the same ability to make a rational decision.

In conclusion, "Can a computer think?" was a tougher question to address than I had thought. I still don't think that I am 100% convinced that a computer can not think but on the other hand I can't say that a computer can think like a human. There is some much more that the human brain can do that we just can't program that into a system. A psychologist from the University of Western Ontario, Zenon Pylyshyn, proposed his own thought thru an ironic experiment: "Thus, if more and more of the cells in your brain were to be replaced by integrated circuit chips, programmed in such a way as to keep the input-output function of each unit identical to the unit being replaced, you would in all likelihood just keep right on speaking exactly as you are doing now except that you would eventually stop meaning anything by it. What we outside observers might take to be words would become for you just certain noises that circuits caused you to make." For now I say that computers can't think but as I learn more about Artificial Intelligence I start to doubt my opinion. I wonder if I will have the same opinion when I finish your (Dr Session's) Artificial Intelligence class.

Work Cited

1. Dyess, Adam <http://library.thinkquest.org/2705/applications.html>
2. Turing, A. M., Computing Machinery and Intelligence
<http://www.loebner.net/Prizef/TuringArticle.html>
3. Waldrop, Mitchell, The Age of Intelligent Machines: Can Computers Think?
<http://www.kurzweilai.net/meme/frame.html?main=/articles/art0103.html>
4. Coppin, Ben Artificial Intelligence Illuminated, Chapter 10: Introduction to machine learning, pg

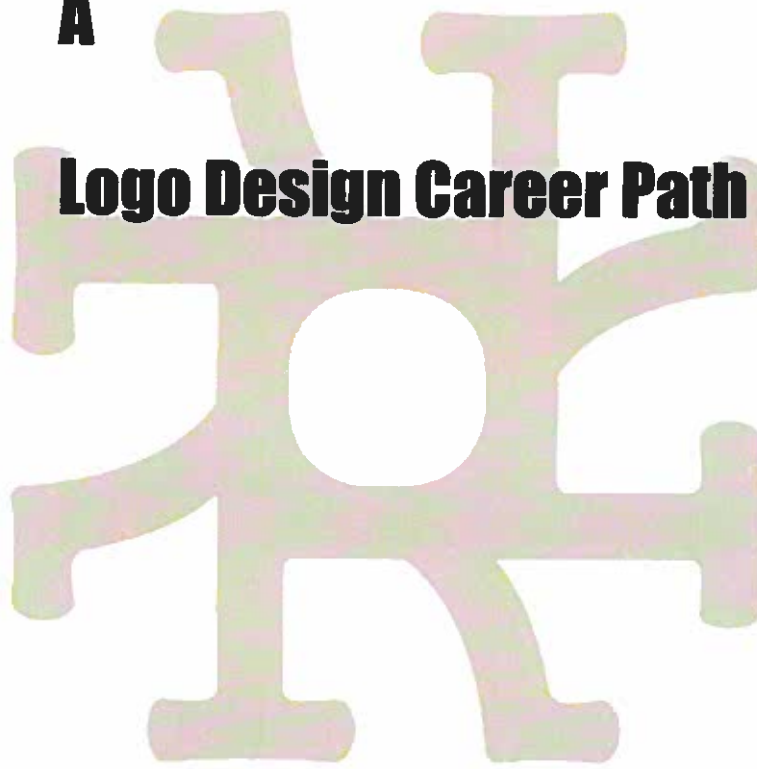
Presentations:

- **COIN495-System Analysis & Design**
- **ARV161-Visual Communication Media**

CLASS: ARV161-Visual Communications
Media

GRADE: A

TOPIC: Logo Design Career Path



presentation

ARV161-VISUAL COMMUNICATIONS MEDIA

Logo Designer career path

Visual Communications Media was a class taken at Trident Technical College where we were required to make a presentation on the career path of a Logo Designer.



presentation

Visual Communications Media

What should you know to be a Logo Designer?

- * Drawing the logo is a tiny part of the job
- * Spend a big part researching
 - Competitors
 - Where will the logo be seen?
 - What are you communicating?
- * Working out all details

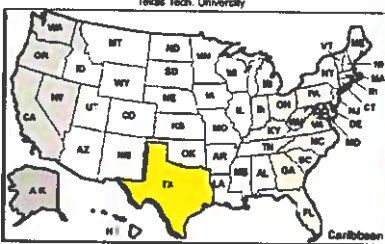


Established Logo Designer Freelance Earning Potential



Client Type	Earning Potential
Individual or Small Firm	\$0 - \$4,000
Small to Medium Companies	\$5,000 - \$10,000
Medium to Large Companies	\$15,000+
Large Corporations	Skies the Limit!

(8) Dallas, TX
(11) Houston, TX
Baylor University
Lamar University
Texas Tech. University



design &
dq *graphics inc.*

IPSI
InvisiWorks, Inc.

MERIDIAN
Builders, Inc.

PCCG

Mystic

The CopyCat

Libertine

Are **YOU** our next potential **LOGO DESIGNER??**

Aaron Boserup

Denise Mathis

Rebecca Randall

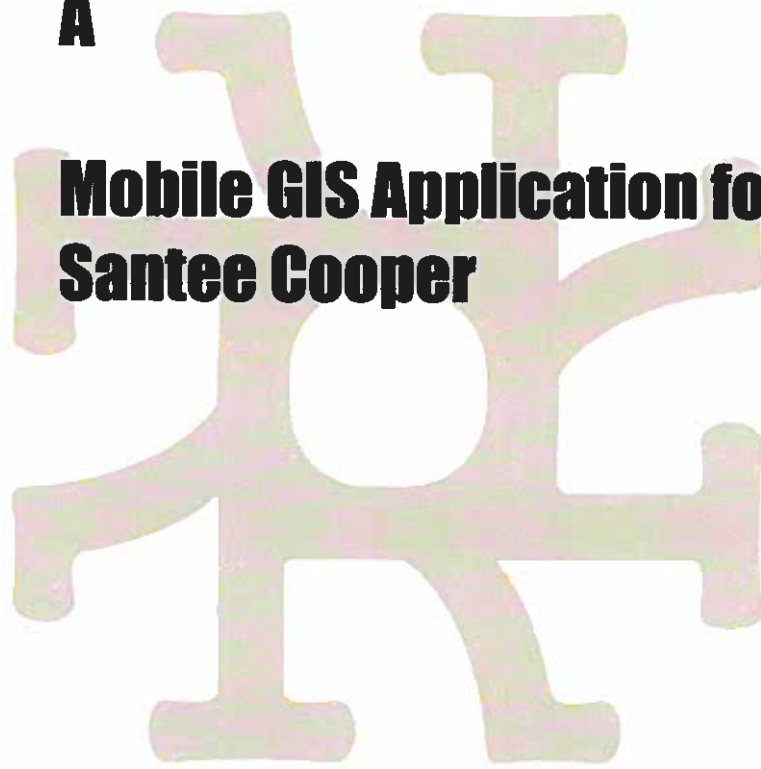
Keep Learning



CLASS: COIN495-System Analysis &
Software Design

GRADE: A

TOPIC: Mobile GIS Application for
Santee Cooper

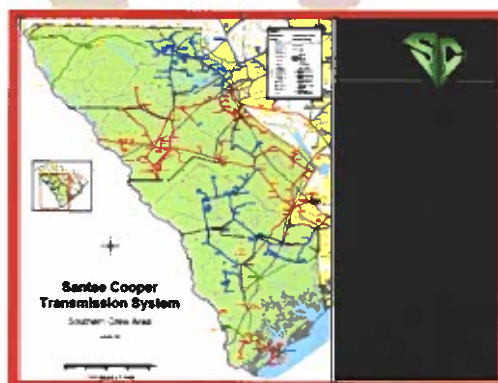
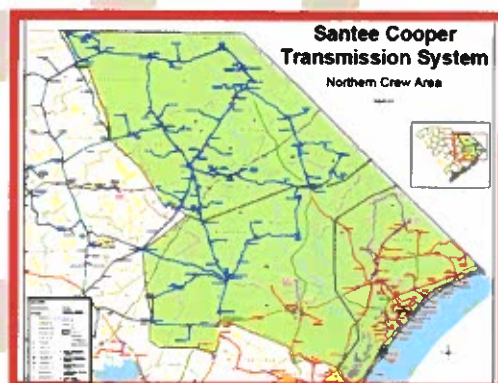
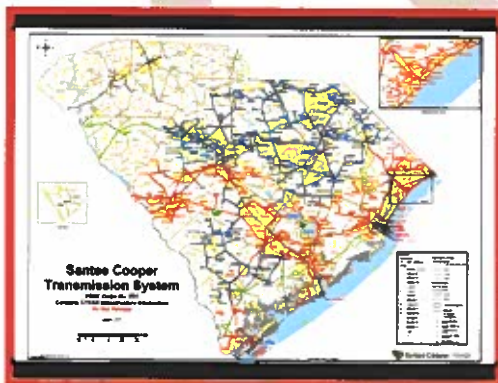
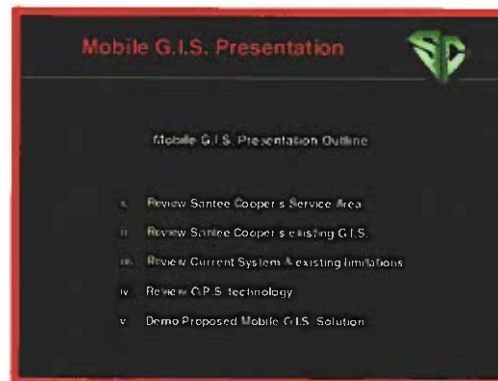
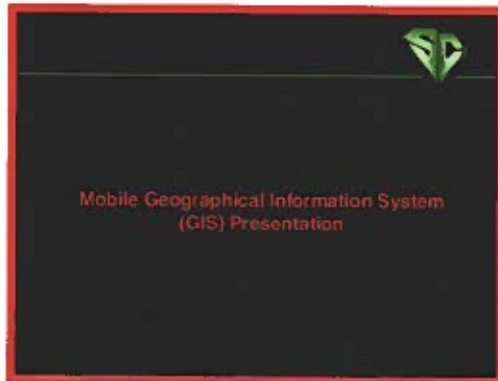


presentation

LEARNING OBJECTIVES: 1. ANALYZE THE PROBLEM 2. DESIGN THE SOLUTION 3. IMPLEMENT THE SOLUTION 4. EVALUATE THE SOLUTION

Mobile Geographical Information System (GIS) Presentation

Our group decided to propose a Mobile GIS application for Santee Cooper field crews to be able to access the GIS data in the field. This presentation was to show the benefits of having such a system and how they would benefit from utilizing a GIS.



presentation



Mobile G.I.S. Presentation

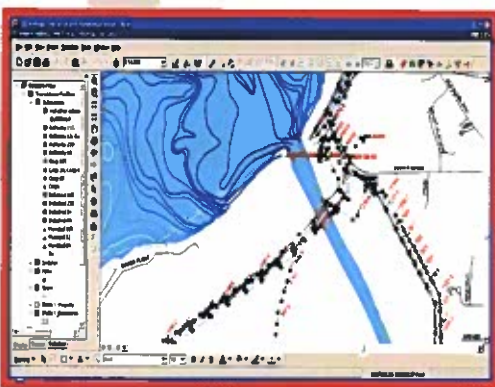
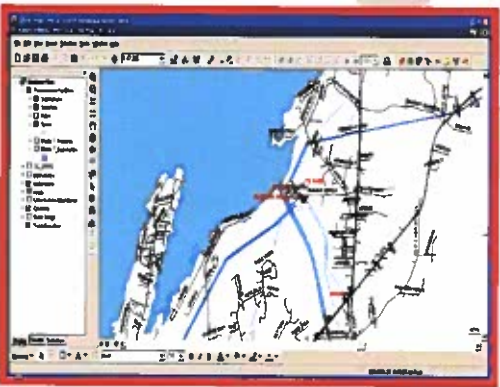
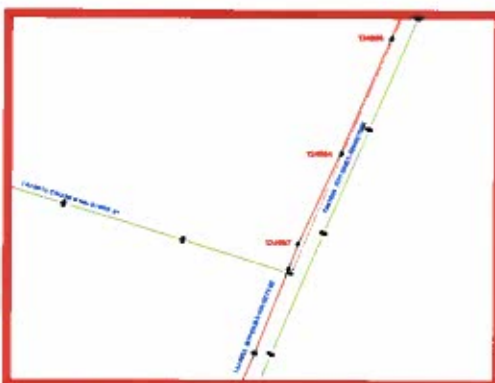
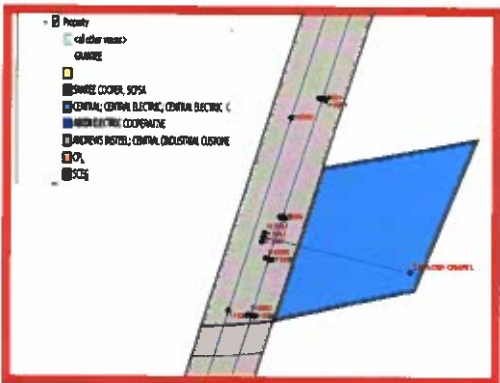
Santee Cooper's GIS program
in Transmission Operations is defined by two
existing systems

T.E.F.I.S.
and
information
system

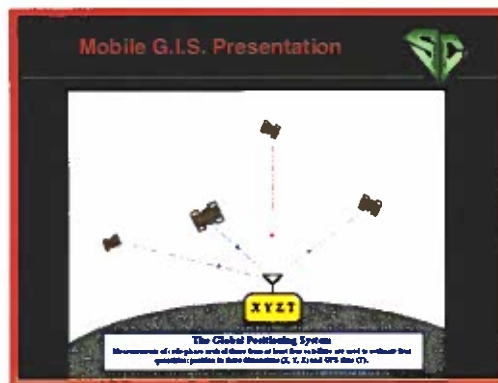
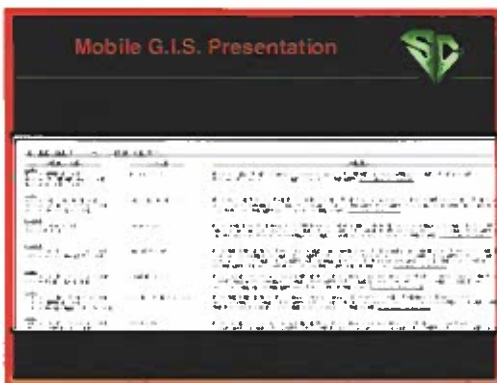
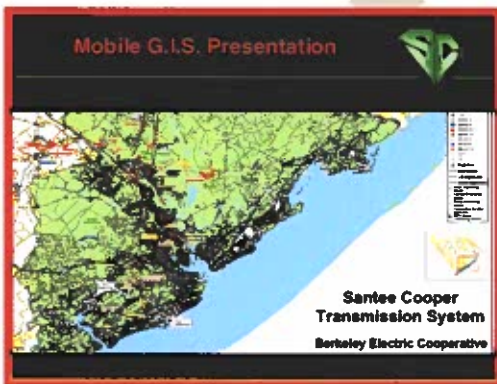
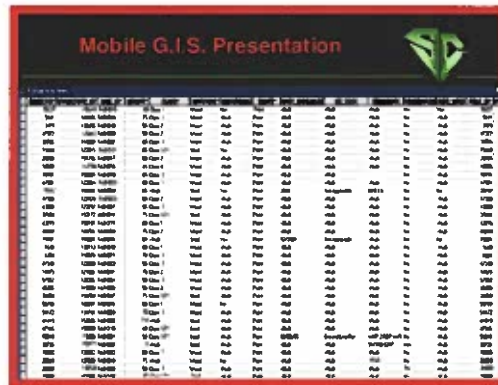
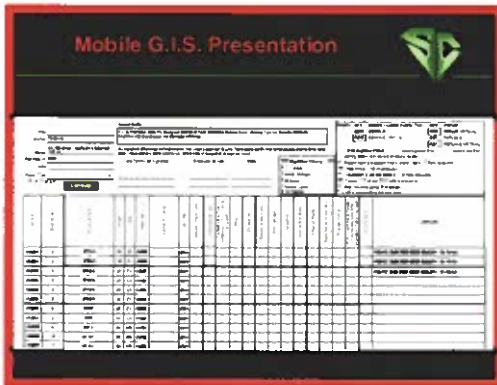
L.I.S. contains transmission elements, SCPSA Substation Property, Co-op Property

Transmission
Electric
Facilities
information
system

T.E.F.I.S. contains transmission spans, poles, switches, and substations.



presentation



presentation

WATER UTILITIES DIVISION
PUBLIC WORKS DEPARTMENT
CITY OF Santee, California

Mobile G.I.S. Presentation



Mobile G.I.S. Presentation



Mobile G.I.S. Presentation



Mobile G.I.S. Presentation Outline

- i. Review Santee Cooper's Service Area
- ii. Review Santee Cooper's existing G.I.S.
- iii. Review Current System & existing limitations
- iv. Review G.P.S. technology
- v. Demo Proposed Mobile G.I.S. Solution